Arcanum: A Dual-Tier Stateless Air-Gapped and Split-VM Architecture for Kleptography-Resistant Zero-Knowledge Proofs

Architectural Specification & Security Model – Version 1.3

Mert Atakan
Conceptual Framework & Protocol Design
ORCID: 0009-0008-4681-7186

M. Zidan Fatonie
Systems Architecture & Verifier Infrastructure
Tawf Labs

Anjul Khatri
Cryptographic Proofs, Circuit Formalization & Split-VM Isolation Architecture

November 2026

Abstract

Zero-Knowledge Proof (ZKP) systems [6] allow a prover to demonstrate knowledge of a secret witness w satisfying a relation $\mathcal{R}(x, w) = 1$ without revealing w to the verifier. However, all standard ZKP toolchains implicitly assume that the proving software stack, compiler pipeline, and underlying host operating system are completely honest and isolated from network exfiltration. In practice, two catastrophic vulnerabilities undermine this assumption: (1) during proof generation, the plaintext witness vector resides exposed in the RAM of network-connected hosts, and (2) a compromised software dependency or malicious circuit compiler can mount a kleptographic (subliminal channel) attack [1, 2, 3], embedding secret witness bits directly inside the mathematical randomness of a valid proof.

In this paper, we present **Arcanum (v1.3)**, an open-source, defense-in-depth execution and sanitization architecture based on the Air-Gapped Zero-Knowledge Proof (AG-ZKP) paradigm. Arcanum introduces a **Dual-Tier Isolation Model** that achieves near-native proving speeds without the $10{,}000\times$ to $100{,}000\times$ computational penalty of Fully Homomorphic Encryption (FHE) [14]. **Tier-1 (Virtual Air-Gap)** isolates the prover on a single physical machine using a networkless Virtual Machine (KVM/QEMU [20] or Qubes OS [21]) hardened via `mlock`, swap elimination, CPU core pinning, Simultaneous Multithreading (SMT) deactivation [24, 25], and optional hardware Trusted Execution Environments (AMD SEV-SNP [22], Intel TDX [23]), connected to a network-facing Sanitizer VM exclusively via a unidirectional write-only channel (`vsock`, `virtio-serial`, or a read-only shared-memory ring buffer). **Tier-2 (Physical Air-Gap)** enforces two-device physical separation via a unidirectional optical data diode (screen-to-camera QR streaming) [26, 27, 28]. Across both tiers, the untrusted Sanitizer enforces **Dual-Track Kleptographic Elimination** [5]: pairing-based proofs (Groth16 [7]) undergo cryptographic proxy rerandomization [8], while hash-based proofs (STARKs/Cairo [10, 13]) enforce Deterministic Fiat-Shamir derivation [15, 16] with channel binding.

Keywords: Zero-Knowledge Proofs, Kleptography, Subliminal Channels, Split-VM Isolation, KVM/QEMU, Qubes OS, Groth16 Rerandomization, Deterministic Fiat-Shamir, STARKs, Optical Data Diode.

Repositories & License: Code licensed under GNU AGPLv3 (`https://github.com/Sidistr/Arcanum` and `https://github.com/Sidistr/ag-zkp`); specification released under CC BY 4.0.

1 Introduction and Threat Landscape

Modern cryptographic protocols rely heavily on Zero-Knowledge Proofs (ZKPs) [6] for verifiable computation, privacy-preserving identity (ZK-KYC), Layer-2 validity rollups (such as Starknet

[10, 13]), and enterprise state verification. While the mathematical soundness and zero-knowledge properties of protocols like Groth16 [7], PLONK [9], and STARKs [10] are rigorously proven in the abstract, their real-world deployments remain vulnerable to system-level and supply-chain compromise.

1.1 The Networked RAM Exposure Problem

During standard proof generation, a circuit compiler (e.g., Circom [12], Cairo [13], Noir) and witness generator must evaluate constraints over the secret input w in plaintext to populate the full witness vector $\mathbf{w} = (1, x_1, \ldots, x_m, w_1, \ldots, w_n)$. On any network-connected machine or conventional cloud server, this witness vector remains exposed in volatile memory, operating system swap files, or crash dumps. Recent enterprise infrastructure failures—such as the industry-wide abandonment of false cloud-managed "air-gap" backup architectures following the ConnectWise management-plane compromises—demonstrate that logical software toggles on network-connected hosts do not constitute true isolation.

1.2 The Supply-Chain Kleptography Problem

Even if a prover machine is placed behind a strict firewall, software supply-chain attacks against package registries (Cargo, npm, PyPI) or precompiled proving binaries introduce a deeper cryptographic threat first formalized by Young and Yung: **kleptography** [1, 2], building upon Simmons' theory of subliminal channels [3] and algorithm substitution attacks (ASAs) [4].

A randomized ZKP protocol relies on internal entropy $r \leftarrow_{\$} \mathcal{E}$ to blind the proof π. A maliciously modified proving library can replace truly random sampling with a deterministic subliminal encoding function:

$$r^* = \mathsf{Encode}_{pk_{\mathsf{adv}}}(w, \mathit{state}) \tag{1}$$

such that the resulting proof $\pi^* = \mathsf{Prove}(pk, x, w; r^*)$ passes the standard verification equation $\mathsf{Verify}(vk, x, \pi^*) = 1$ with probability 1, yet allows an external adversary observing π^* on a public blockchain or verification server to extract the secret witness w [1, 4].

1.3 Limitations of Existing Countermeasures

Existing approaches fail to simultaneously resolve networked memory exposure, supply-chain kleptography, and performance feasibility, as summarized in Table 1.

2 Arcanum Dual-Tier Isolation Architecture

To serve both high-throughput automated enterprise/consumer environments and sovereign defense-grade deployments without forcing all users to purchase secondary hardware, Arcanum v1.3 defines a unified two-tier isolation architecture.

2.1 Tier-1: Single-Machine Split-VM Architecture (Virtual Air-Gap)

Designed for single-machine deployments, developer workstations, enterprise server racks, and privacy-focused desktop environments, Tier-1 partitions a single physical host into two strictly separated virtual domains using KVM/QEMU [20] or Qubes OS [21]:

Approach	Host & Side-Channel Risk	Kleptographic Resistance	Performance Overhead
Standard ZKP Stack	Critical (Plaintext witness in networked RAM/swap)	None (Subliminal channels in proof randomness [1, 3])	$1\times$ (Baseline)
Standalone Hardware TEE	High (Vulnerable to L1/L2 cache & microarchitectural attacks [24, 25])	None (Malicious enclave code leaks via valid proof bytes)	$1.2\times$ –$3\times$
Fully Homomorphic (FHE) [14]	Low (Encrypted evaluation)	Partial (Ciphertext metadata channels remain)	$10{,}000\times$ –$100{,}000\times$
Arcanum (Tier-1 Split-VM)	Mitigated (no-NIC, `mlock`, CPU pinning, SMT off, TEE [20, 22, 23])	Full (Sanitizer VM Rerandomization / Det. FS [5, 8])	$1.05\times$ –$1.15\times$
Arcanum (Tier-2 Optical)	Eliminated (Physical two-device air-gap + QR diode [26, 28])	Full (External Proxy Sanitization + Channel Bind)	$1.05\times$ + QR stream

Table 1: Comparison of Arcanum Dual-Tier Architecture against existing privacy and isolation paradigms.

1. **Networkless Prover VM (no-NIC Domain):** The proving engine executes inside an isolated Virtual Machine provisioned with zero virtual or physical network interfaces. The Prover VM cannot initiate or receive TCP/IP, UDP, or socket connections to the outside world.

2. **Circuit & Witness Execution Pipeline (Circom / R1CS / STARK Flow):** To maintain a minimal and auditable attack surface inside the isolated environment, circuit compilation is decoupled from witness evaluation:

   - Compiled circuit constraints—such as Rank-1 Constraint System (R1CS) binaries [12] or arithmetic execution traces [13]—along with the deterministic witness generation code are exported from the build environment and mounted inside the Prover VM as a read-only image.
   - Private inputs w are supplied exclusively inside the Prover VM (stored on an encrypted disk volume attached solely to that VM).
   - The witness generator populates the witness vector $\mathbf{w} \in \mathbb{F}_p^m$ satisfying $A\mathbf{w} \circ B\mathbf{w} = C\mathbf{w}$ and passes $\mathbf{w}$ directly in locked memory to the proving engine to produce the raw proof bytes π_{raw}.

3. **Unidirectional Write-Only Channel to the Sanitizer VM:** A second Virtual Machine—the **Sanitizer VM (Untrusted Proxy)**—holds the external network interface. Communication from the Prover VM to the Sanitizer VM is restricted to a strict one-way, write-only data path implemented via one of three hypervisor primitives:

   - A unidirectional `vsock` or `virtio-serial` character device configured for write-only transmission from the Prover VM; or
   - A shared-memory ring buffer (`ivshmem`) mapped with write permissions on the Prover VM and strictly read-only page table permissions on the Sanitizer VM side.

   The Prover VM writes the serialized raw proof bytes (x, π_{raw}) into the ring buffer. The Sanitizer VM reads the raw bytes, verifies mathematical validity, applies cryptographic sanitization (Section 3), and publishes the sanitized proof π' to the external network or blockchain.

4. **Memory & Microarchitectural Hardening:** Because witness allocation in standard runtime environments can leave residual artifacts in physical RAM or OS page files, the Prover VM enforces four mandatory hardening controls:

    - *Swap Elimination:* OS swap is completely disabled (or enforced via ephemeral random-key encrypted swap) so plaintext witness vectors never touch persistent storage.
    - *Physical RAM Locking & Zeroization:* Sensitive witness and proving buffers are pinned in physical memory via `mlock()` to prevent hypervisor paging, core dumps (`RLIMIT_CORE = 0`) are disabled, and deterministic memory scrubbing (`zeroize`) overwrites **w** immediately after proof emission.
    - *CPU Core Pinning & SMT Deactivation:* The Prover VM vCPUs are pinned to dedicated physical CPU cores (`vcpupin`), and Simultaneous Multithreading (SMT/Hyper-Threading) is disabled to prevent cross-VM L1/L2/L3 cache timing and speculative execution side-channel extraction [24, 25].
    - *Confidential Computing (Optional TEE Encapsulation):* On supported enterprise hardware, the Prover VM is encapsulated inside a hardware-enforced Trusted Execution Environment (AMD SEV-SNP [22] or Intel TDX [23]), encrypting VM memory pages so that even a compromised host OS or root hypervisor cannot inspect the prover's RAM.

2.2 Tier-2: Two-Device Physical Air-Gap (Optical Data Diode)

For sovereign root key ceremonies, military/B2G deployments, and high-value cold vaults where sharing a single physical motherboard is unacceptable under TEMPEST or electromagnetic emission mandates [28], Tier-2 physically separates the Prover and the Sanitizer onto two independent hardware devices:

- **Stateless Air-Gapped Prover Device:** Runs on stripped-down hardware with physically removed wireless/Ethernet controllers and ephemeral RAMdisk storage.
- **Unidirectional Optical Data Diode:** Raw proof payloads are encoded into a fountain-coded (Luby Transform [26] / Reed-Solomon [27]) sequence of high-density animated QR frames rendered on the Prover screen and captured by the Sanitizer's optical camera. Because photons travel strictly from screen to lens, zero backward electromagnetic or software return channel exists.

3 Dual-Track Kleptographic Channel Elimination

Isolating the Prover VM or physical device prevents direct network exfiltration, but does not by itself stop a compromised proving binary from hiding witness bits inside the valid proof π_{raw} sent through the ring buffer or optical diode [1, 4]. Acting as a cryptographic reverse firewall [5], Arcanum eliminates subliminal channels inside the Sanitizer VM across both major classes of ZKP systems.

3.1 Track A: Untrusted Proxy Rerandomization for Pairing-Based SNARKs (Groth16)

In Groth16 [7] over a bilinear pairing group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, a raw proof emitted by the Prover VM consists of three group elements:

$$\pi_{\text{raw}} = ([A]_1, [B]_2, [C]_1) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1 \tag{2}$$

Even if a supply-chain trojan inside the Prover VM manipulates the internal blinding scalars $(r, s) \in \mathbb{F}_p^2$ to encode secret bits of the witness w, the Sanitizer VM intercepts π_{raw} before network broadcast:

1. **Pre-Sanitization Verification:** The Sanitizer checks that π_{raw} satisfies the Groth16 bilinear pairing equation for public statement x:

$$e([A]_1, [B]_2) = e([\alpha]_1, [\beta]_2) \cdot e([L_{\text{pub}}(x)]_1, [\gamma]_2) \cdot e([C]_1, [\delta]_2) \tag{3}$$

   and verifies subgroup membership in $\mathbb{G}_1$ and $\mathbb{G}_2$. If verification fails, the payload is dropped immediately.

2. **Fresh Entropy Injection:** Following randomizable proof constructions [8, 5], the Sanitizer samples two independent, cryptographically secure random scalars $r', s' \leftarrow_{\$} \mathbb{F}_p^*$ from its own entropy pool (with zero knowledge of w) and computes the rerandomized proof $\pi' = ([A']_1, [B']_2, [C']_1)$:

$$[A']_1 = r' \cdot [A]_1 \tag{4}$$
$$[B']_2 = (r')^{-1} \cdot [B]_2 + r's' \cdot [\delta]_2 \tag{5}$$
$$[C']_1 = [C]_1 + s' \cdot [A]_1 \tag{6}$$

Theorem 1 (Information-Theoretic Subliminal Channel Destruction): *For any valid Groth16 proof π_{raw} generated by an arbitrarily malicious Prover VM for statement x, the rerandomized proof π' produced by the Sanitizer VM is uniformly distributed over the space of all valid proofs for x, and the mutual information between the private witness W and the published proof Π' conditioned on the public statement X is strictly zero:*

$$I(W; \Pi' \mid X = x) = 0 \tag{7}$$

Proof Sketch: Because $r' \in \mathbb{F}_p^*$ acts as a multiplicative one-time pad in the prime-order group $\mathbb{G}_1$ (assuming $[A]_1 \neq \mathcal{O}$), $[A']_1$ is uniformly distributed in $\mathbb{G}_1 \setminus \{\mathcal{O}\}$ independent of $[A]_1$. Similarly, $s' \in \mathbb{F}_p^*$ additively blinds $[B']_2$ along $[\delta]_2$, while $[C']_1$ is uniquely determined by the bilinear verification equation. Thus, every kleptographic bit encoded in (r, s) is algebraically obliterated [7, 8, 5]. ■

3.2 Track B: Deterministic Fiat-Shamir & Channel Binding for STARKs and L2 Verifiers

Hash-based Interactive Oracle Proofs (STARKs [10], FRI [11], and Starknet Cairo proofs [13]) lack algebraic group homomorphisms and therefore cannot be externally rerandomized by an untrusted proxy. In standard STARK provers, optional salt grinding and non-deterministic trace padding allow a compromised prover to grind commitments until low-order bits leak secret witness bytes.

Arcanum secures STARK and Cairo proving pipelines through three deterministic constraints:

1. **RFC 6979-Style Deterministic Seed Derivation:** Inspired by deterministic digital signature generation [15, 16], all prover randomness ρ_{det} (including FRI blinding and polynomial masking) is derived deterministically via a cryptographic sponge (Poseidon [17], Rescue-Prime [18], or SHA-3 [19]) from the canonical witness commitment and public inputs:

$$\rho_{\text{det}} = \text{Sponge}(\text{DomSep} \parallel \text{Commit}(w) \parallel x \parallel \text{ChannelID}) \tag{8}$$

2. **Zero-Grinding Verification in the Sanitizer VM:** All variable proof-of-work or salt fields in the Fiat-Shamir transcript [15] are pinned to canonical zero constants ($salt = 0^{256}$) or bound directly to the Sanitizer's challenge nonce.
3. **Dual-Modular Redundancy (DMR) Watchdog:** Inside the Prover VM (or across two independent compiler builds, e.g., GCC and Clang), deterministic execution ensures that two honest runs produce bit-for-bit identical proof bytes: $\pi_{\mathsf{raw}}^{(1)} \equiv \pi_{\mathsf{raw}}^{(2)}$. Any divergence induced by a non-deterministic trojan immediately triggers an abort in the Sanitizer VM before network publication.

4 Ecosystem Applications and Target Segments

By combining Tier-1 (Single-Machine Split-VM) and Tier-2 (Physical Optical Diode), Arcanum serves three high-impact deployment segments:

4.1 Enterprise Verifiable Backup, Cloud Infrastructure & DeFi (B2B via Split-VM + TEE)

Automated data centers and institutional provers cannot rely on manual optical cameras. Using Arcanum's Tier-1 KVM/QEMU Split-VM architecture [20] with shared-memory ring buffers and AMD SEV-SNP [22] / Intel TDX [23], enterprise backup servers generate periodic zero-knowledge proofs of storage integrity and ransomware-free state transitions without exposing backup decryption keys or witness state to the network-facing management plane.

4.2 Everyday Privacy: Kleptography-Resistant ZK-KYC & Delegated Anonymous Spending (B2C via Split-VM)

Regular users require strong isolation on a single personal computer without secondary hardware:

- **Kleptography-Resistant ZK-KYC:** Users prove passport, age, or credential validity locally inside the networkless Prover VM. Even if a third-party identity circuit contains a supply-chain backdoor [1, 4], the Sanitizer VM strips any subliminal leakage [5] so biometric or identity numbers cannot be exfiltrated inside the proof.
- **Delegated Anonymous Spending Protocol (Cryptographically Controlled Allowance Pools):** Arcanum integrates a privacy-preserving on-chain allowance protocol utilizing proxy signatures [29, 30] and zero-knowledge policy proofs [8]. A Master Delegator locks funds into a cryptographically controlled pool and generates delegated proxy credentials enforcing granular spending policies (e.g., rate limits, merchant categories, or epoch caps). By isolating the Master Delegator key inside the Arcanum Prover VM while proxy users execute anonymous withdrawals on-chain, the delegator retains full cryptographic authority to withdraw pool funds or revoke any compromised proxy without ever exposing the master secret to network-connected wallet software.

4.3 Sovereign Public Sector, Defense & Cold Treasury Vaults (B2G via Optical Diode & Qubes OS)

For national critical infrastructure, military command verification, and multi-million-dollar institutional cold treasuries, Arcanum provides Tier-2 two-device Optical Data Diode isolation

[26, 27, 28] alongside native Qubes OS Split-VM templates [21] for field operators.

5 Implementation Roadmap

1. **Phase 1 – Split-VM Reference Testbed & Circom/R1CS Integration:** Packaging reproducible KVM/QEMU [20] and Qubes OS [21] VM descriptors, read-only R1CS/witness import pipelines [12], `vsock`/shared-memory ring buffer transport, and Linux memory hardening (`mlock`, `zeroize`, swap suppression, CPU pinning).
2. **Phase 2 – Cryptographic Sanitizer Engine (Groth16 & STARKs):** Rust implementation of the Sanitizer VM proxy rerandomizer [7, 8, 5] over BN254 and BLS12-381 curves (`arkworks`) and Deterministic Fiat-Shamir transcript verification [15, 16] for STARK/Cairo provers [10, 13].
3. **Phase 3 – Optical Data Diode & Delegated Anonymous Spending Modules:** Unidirectional QR fountain-code streaming library [26, 27] and reference circuits for ZK-KYC and Delegated Anonymous Spending pools [29, 30].

References

[1] A. Young and M. Yung, "The Dark Side of Black-Box Cryptography, or: Should We Trust Capstone?" in *Advances in Cryptology – CRYPTO '96*, Lecture Notes in Computer Science, vol. 1109, Springer, 1996, pp. 89–103.

[2] A. Young and M. Yung, *Malicious Cryptography: Exposing Cryptovirology*, John Wiley & Sons, 2004.

[3] G. J. Simmons, "The Prisoners' Problem and the Subliminal Channel," in *Advances in Cryptology – Proceedings of Crypto 83*, Plenum Press, 1984, pp. 51–67.

[4] M. Bellare, K. G. Paterson, and P. Rogaway, "Security of Symmetric Encryption against Mass Surveillance," in *Advances in Cryptology – CRYPTO 2014*, Lecture Notes in Computer Science, vol. 8616, Springer, 2014, pp. 1–19.

[5] I. Mironov and N. Stephens-Davidowitz, "Cryptographic Reverse Firewalls," in *Advances in Cryptology – EUROCRYPT 2015*, Lecture Notes in Computer Science, vol. 9057, Springer, 2015, pp. 657–686.

[6] S. Goldwasser, S. Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," *SIAM Journal on Computing*, vol. 18, no. 1, 1989, pp. 186–208.

[7] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," in *Advances in Cryptology – EUROCRYPT 2016*, Lecture Notes in Computer Science, vol. 9666, Springer, 2016, pp. 305–326.

[8] M. Belenkiy, J. Camenisch, M. Chase, M. Kohlweiss, A. Lysyanskaya, and H. Shacham, "Randomizable Proofs and Delegatable Anonymous Credentials," in *Advances in Cryptology – CRYPTO 2009*, Lecture Notes in Computer Science, vol. 5677, Springer, 2009, pp. 108–125.

[9] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR Cryptology ePrint Archive*, Report 2019/953, 2019.

[10] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, Transparent, and Post-Quantum Secure Computational Integrity,” *IACR Cryptology ePrint Archive*, Report 2018/046, 2018.

[11] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, LIPIcs, vol. 107, 2018, pp. 14:1–14:17.

[12] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A Circuit Description Language for Building Zero-Knowledge Applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, 2023, pp. 4733–4751.

[13] L. Goldberg, S. Papini, and M. Riabzev, “Cairo – A Turing-Complete STARK-Friendly CPU Architecture,” *IACR Cryptology ePrint Archive*, Report 2021/1063, 2021.

[14] C. Gentry, “Fully Homomorphic Encryption Using Ideal Lattices,” in *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC ’09)*, 2009, pp. 169–178.

[15] A. Fiat and A. Shamir, “How to Prove Yourself: Practical Solutions to Identification and Signature Problems,” in *Advances in Cryptology – CRYPTO ’86*, Lecture Notes in Computer Science, vol. 263, Springer, 1987, pp. 186–194.

[16] T. Pornin, “Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA),” *Internet Engineering Task Force (IETF)*, RFC 6979, August 2013.

[17] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 519–535.

[18] A. Aly, T. Ashur, E. Ben-Sasson, S. Dhooghe, and A. Szepieniec, “Design of Symmetric-Key Primitives for Advanced Cryptographic Protocols,” *IACR Transactions on Symmetric Cryptology (ToSC)*, vol. 2020, no. 3, 2020, pp. 1–45.

[19] National Institute of Standards and Technology (NIST), “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” *Federal Information Processing Standards Publication (FIPS) 202*, August 2015.

[20] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, “kvm: the Linux Virtual Machine Monitor,” in *Proceedings of the Linux Symposium*, vol. 1, 2007, pp. 225–230.

[21] J. Rutkowska and R. Wojtczuk, “Qubes OS Architecture,” *Invisible Things Lab Technical Report*, 2010.

[22] Advanced Micro Devices (AMD), “AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More,” *AMD White Paper*, January 2020.

[23] Intel Corporation, “Intel Trust Domain Extensions (Intel TDX),” *Intel White Paper*, 2020.

[24] Y. Yarom and K. Falkner, “FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 719–732.

[25] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, “Spectre Attacks: Exploiting Speculative Execution,” in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1–19.

[26] M. Luby, “LT Codes,” in *Proceedings of the 43rd Annual IEEE Symposium on Foundations of Computer Science (FOCS ’02)*, 2002, pp. 271–280.

[27] I. S. Reed and G. Solomon, “Polynomial Codes Over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics (SIAM)*, vol. 8, no. 2, 1960, pp. 300–304.

[28] M. Guri, G. Kedma, A. Kachlon, and Y. Elovici, “AirHopper: Bridging the Air-Gap between Isolated Networks and Mobile Phones using Radio Frequencies,” in *9th IEEE International Conference on Malicious and Wanted Software (MALWARE)*, 2014, pp. 58–67.

[29] M. Mambo, K. Usuda, and E. Okamoto, “Proxy Signatures: Delegation of the Power to Sign Messages,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E79-A, no. 9, 1996, pp. 1338–1354.

[30] A. Boldyreva, A. Palacio, and B. Warinschi, “Secure Proxy Signature Schemes for Delegation of Signing Rights,” *Journal of Cryptology*, vol. 25, no. 1, 2012, pp. 57–115.